

Основы Python для робототехники

Введение

Перед тем как начать программировать роботов, тебе нужно изучить основы Python. Не волнуйся - тебе нужно выучить всего несколько простых вещей, чтобы начать!

Эта глава научит тебя именно тому, что нужно для управления роботами. Ни больше, ни меньше.

Что ты узнаешь: - Как писать и запускать программы на Python - Переменные (хранение информации) - Вывод текста (показ сообщений) - Математика (вычисления) - Условия if (принятие решений) - Циклы (повторение действий) - Функции (переиспользование кода) - Импорт (использование готовых инструментов)

Время: 1-2 часа

Твоя первая программа на Python

Создание файла Python

Программы на Python - это текстовые файлы с расширением .py. Давай создадим такой файл:

```
cd ~  
nano hello.py
```

Напиши это:

```
print("Привет, Робот!")
```

Сохрани файл (Ctrl+O, Enter, Ctrl+X).

Запуск программы

```
python3 hello.py
```

Ты должен увидеть:

```
Привет, Робот!
```

Поздравляю! Ты только что написал свою первую программу на Python!

Комментарии - заметки для себя

Комментарии - это заметки в твоём коде. Python их игнорирует - они нужны только тебе, чтобы помнить, что делает код.

```
# Это комментарий - Python игнорирует эту строку
```

```
print("Привет") # Можно писать комментарии и после кода
```

Используй комментарии, чтобы: - Объяснить, что делает твой код - Напомнить себе, зачем ты что-то сделал - Помочь другим понять твой код

Пример:

```
# Запустить мотор
motor.start()

# Вращать со скоростью 30 об/мин в течение 3 секунд
motor.run(30)
time.sleep(3)

# Остановить и завершить
motor.stop()
```

Переменные - хранение информации

Переменные - это как коробочки, в которых ты храни информацию.

Создание переменных

```
speed = 30
name = "Робот"
is_running = True
```

Правила для имён переменных: - Можно использовать буквы, цифры, подчёркивание (_) - Должны начинаться с буквы - БЕЗ пробелов (используй подчёркивание вместо них) - Регистр важен: Speed и speed - это разные переменные!

Хорошие имена:

```
motor_speed = 30
robot_name = "Ева"
max_distance = 100
```

Плохие имена:

```
s = 30 # Слишком короткое - что означает 's'?
motor speed = 30 # НЕТ! Есть пробел
123speed = 30 # НЕТ! Начинается с цифры
```

Использование переменных

```
speed = 30
print(speed) # Выведет: 30

motor.run(speed) # Используем переменную вместо того, чтобы писать 30
```

Изменение переменных

```
speed = 30
print(speed)      # Выведет: 30

speed = 60
print(speed)      # Выведет: 60
```

Числа и математика

Типы чисел

Целые числа (integers):

```
age = 10
speed = 30
count = 5
```

Дробные числа (floats):

```
distance = 3.5
temperature = 98.6
pi = 3.14159
```

Математические операции

```
# Сложение
result = 5 + 3      # result равно 8

# Вычитание
result = 10 - 4     # result равно 6

# Умножение
result = 3 * 4       # result равно 12

# Деление
result = 10 / 2      # result равно 5.0 (всегда даёт дробное число!)

# Целочисленное деление (без дробной части)
result = 10 // 3     # result равно 3

# Остаток от деления (модуло)
result = 10 % 3      # result равно 1 (10 ÷ 3 даёт остаток 1)
```

Использование математики с переменными

```
speed = 30
faster = speed + 10  # faster равно 40
slower = speed - 10  # slower равно 20
double = speed * 2   # double равно 60
```

```
half = speed / 2          # half равно 15.0
```

Распространённая математика для роботов:

```
# Перевод градусов в импульсы
degrees = 90
pulses = degrees * 8.889    # pulses равно 800.01

# Перевод об/мин в обороты в секунду
rpm = 60
rps = rpm / 60              # rps равно 1.0

# Расчёт общего расстояния
distance_per_rotation = 0.25 # метры
rotations = 4
total_distance = distance_per_rotation * rotations # 1.0 метр
```

Вывод текста - показ информации

Базовый вывод

```
print("Привет")           # Выведет: Привет
print(42)                  # Выведет: 42
print(3.14)                # Выведет: 3.14
```

Вывод переменных

```
speed = 30
print(speed)               # Выведет: 30

name = "Робот"
print(name)                # Выведет: Робот
```

Вывод нескольких значений

```
speed = 30
print("Скорость:", speed)  # Выведет: Скорость: 30
print("Скорость", speed, "об/мин") # Выведет: Скорость 30 об/мин
```

Ф-строки (современный способ)

Это ЛУЧШИЙ способ выводить переменные:

```
speed = 30
print(f"Скорость: {speed} об/мин")    # Выведет: Скорость: 30 об/мин

name = "Ева"
age = 10
print(f"{name} {age} лет")            # Выведет: Ева 10 лет
```

Почему f-строки классные: - Легко читать - Переменные вставляются прямо там, где нужно - Можно делать вычисления внутри них

```
speed = 30
print(f"Двойная скорость: {speed * 2}") # Выведет: Двойная скорость: 60
```

Форматирование чисел

```
# Показать 2 знака после запятой
distance = 3.14159
print(f"Расстояние: {distance:.2f} метров") # Выведет: Расстояние: 3.14 метров

# Показать как целое число (без дробной части)
value = 123.456
print(f"Значение: {value:.0f}") # Выведет: Значение: 123
```

Отступы - ОЧЕНЬ ВАЖНО!

Python использует **отступы** (пробелы в начале строк), чтобы группировать код. Это ОЧЕНЬ важно и часто вызывает проблемы у новичков!

Что такое отступы?

```
# Без отступа - это обычный код
print("Начало")

# С отступом - это код принадлежит чему-то
    print("Это с отступом")
```

Когда делать отступы

Ты делаешь отступ для кода, который находится "внутри" чего-то:

```
# Условие if
if speed > 50:
    print("Слишком быстро!") # С отступом - принадлежит 'if'
    speed = 50               # С отступом - принадлежит 'if'

# Цикл
for i in range(3):
    print(i)                 # С отступом - принадлежит 'for'
    motor.run(30)            # С отступом - принадлежит 'for'

# Функция
def start_motor():
    motor.start()            # С отступом - принадлежит функции
    motor.run(30)            # С отступом - принадлежит функции
```

Сколько делать отступ?

Используй 4 пробела (или одну клавишу Tab). Выбери один вариант и придерживайся его!

```
# Хорошо (4 пробела)
if speed > 50:
    print("Быстро")

# Хорошо (1 tab)
if speed > 50:
    print("Быстро")

# ПЛОХО (смешивание пробелов и табуляции)
if speed > 50:
    print("Строка 1")      # 4 пробела
    print("Строка 2")      # 1 tab
# Python выдаст ошибку!
```

Частые ошибки с отступами

Ошибка 1: забыл сделать отступ

```
# НЕПРАВИЛЬНО
if speed > 50:
print("Слишком быстро")      # ОШИБКА! Нужен отступ

# ПРАВИЛЬНО
if speed > 50:
    print("Слишком быстро")    # С отступом!
```

Ошибка 2: лишний отступ

```
# НЕПРАВИЛЬНО
print("Начало")
    print("Дальше")           # ОШИБКА! Зачем отступ?

# ПРАВИЛЬНО
print("Начало")
print("Дальше")               # Отступ не нужен
```

Ошибка 3: непостоянные отступы

```
# НЕПРАВИЛЬНО
if speed > 50:
    print("Строка 1")         # 4 пробела
    print("Строка 2")         # 6 пробелов - ОШИБКА!

# ПРАВИЛЬНО
if speed > 50:
    print("Строка 1")         # 4 пробела
    print("Строка 2")         # 4 пробела
```

Условия if - принятие решений

Условия if позволяют твоей программе принимать решения.

Базовое if

```
speed = 60

if speed > 50:
    print("Едем быстро!")
```

Это проверяет: "Скорость больше 50?" Если да, выводим сообщение.

If-Else

```
speed = 30

if speed > 50:
    print("Едем быстро!")
else:
    print("Едем медленно")
```

Если скорость больше 50, выводим "Едем быстро!". Иначе выводим "Едем медленно".

If-Elif-Else

```
speed = 45

if speed > 60:
    print("Очень быстро!")
elif speed > 30:
    print("Средняя скорость")
else:
    print("Медленно")
```

Как это работает: 1. Проверяем `speed > 60` → Нет? Пробуем дальше... 2. Проверяем `speed > 30` → Да! Выводим "Средняя скорость" и останавливаемся 3. Если ничего не подошло, выполняем `else`

Операторы сравнения

```
speed = 50

# Равно
if speed == 50:
    print("Ровно 50!")

# Не равно
if speed != 30:
    print("Не 30")
```

```

# Больше чем
if speed > 40:
    print("Больше 40")

# Меньше чем
if speed < 60:
    print("Меньше 60")

# Больше или равно
if speed >= 50:
    print("50 или больше")

# Меньше или равно
if speed <= 50:
    print("50 или меньше")

```

Частая ошибка:

```

# НЕПРАВИЛЬНО - используется один =
if speed = 50:          # ОШИБКА!

# ПРАВИЛЬНО - используется двойной ==
if speed == 50:         # Проверка на равенство

```

Комбинирование условий

```

speed = 40
distance = 100

# AND - оба должны быть истинны
if speed > 30 and distance > 50:
    print("Быстро и далеко!")

# OR - хотя бы одно должно быть истинным
if speed > 60 or distance > 200:
    print("Либо быстро, либо очень далеко!")

# NOT - противоположное
if not speed > 50:
    print("Не едем быстро")

```

Примеры с роботом

```

# Проверка, двигался ли мотор
position = motor.get_position()
if position > 3200:
    print("Мотор прошёл больше 1 оборота!")

# Проверка безопасности
if speed > 100:
    print("Внимание: скорость слишком высокая!")
    speed = 100

```

```
# Проверка нескольких моторов
if motor1.get_position() > 1000 and motor2.get_position() > 1000:
    print("Оба мотора сдвинулись!")
```

Циклы - повторение действий

Циклы позволяют тебе повторять код несколько раз.

Цикл For с Range

Повторить что-то определённое количество раз:

```
# Вывести числа от 0 до 4
for i in range(5):
    print(i)
```

```
# Вывод:
# 0
# 1
# 2
# 3
# 4
```

Как работает range():

```
range(5)          # 0, 1, 2, 3, 4 (5 чисел, начиная с 0)
range(1, 6)       # 1, 2, 3, 4, 5 (начать с 1, остановиться перед 6)
range(0, 10, 2)   # 0, 2, 4, 6, 8 (начать с 0, остановиться перед 10, шаг 2)
```

Примеры:

```
# Считать до 10
for i in range(1, 11):
    print(i)
```

```
# Считать десятками
for i in range(10, 101, 10):
    print(i)          # Выведет: 10, 20, 30, ... 100
```

```
# Обратный отсчёт
for i in range(5, 0, -1):
    print(i)          # Выведет: 5, 4, 3, 2, 1
```

Цикл For со списками

Проходить по элементам в списке:

```
motors = [motor1, motor2, motor3]

for motor in motors:
    motor.start()
```

Больше примеров:

```
# Цикл по числам
speeds = [10, 20, 30, 40]
for speed in speeds:
    print(f"Скорость: {speed}")

# Цикл по именам
names = ["Ева", "Валли", "ВВ-8"]
for name in names:
    print(f"Привет, {name}!")
```

Цикл While

Повторять, пока условие истинно:

```
count = 0

while count < 5:
    print(count)
    count = count + 1

# Вывод: 0, 1, 2, 3, 4
```

Пример с роботом:

```
# Продолжать движение, пока не пройдем 10000 импульсов
while motor.get_position() < 10000:
    motor.run(30)
    time.sleep(0.1)
motor.stop()
```

Предупреждение: Будь осторожен с циклами while! Если условие никогда не станет False, цикл будет работать вечно!

```
# ПЛОХО - работает вечно!
while True:
    print("Никогда не остановится!")

# ХОРОШО - есть способ остановиться
count = 0
while count < 10:
    print(count)
    count = count + 1 # В конце концов count станет >= 10
```

Break и Continue

Break - выйти из цикла немедленно:

```
for i in range(100):
    if i == 5:
```

```
        break          # Остановить цикл, когда i равно 5
    print(i)

# Вывод: 0, 1, 2, 3, 4
```

Continue - пропустить на следующую итерацию:

```
for i in range(5):
    if i == 2:
        continue      # Пропустить, когда i равно 2
    print(i)

# Вывод: 0, 1, 3, 4 (пропустили 2)
```

Примеры с роботом

```
# Постепенно ускоряться
for speed in range(10, 101, 10):
    motor.run(speed)
    time.sleep(1)

# Запустить все моторы
motors = [motor1, motor2, motor3, motor4]
for motor in motors:
    motor.start()

# Повторить движение 5 раз
for i in range(5):
    motor.run(30)
    time.sleep(2)
    motor.hold()
    time.sleep(1)
```

Функции - переиспользование кода

Функции - это переиспользуемые куски кода. Вместо того, чтобы писать один и тот же код снова и снова, напиши его один раз в функции!

Создание функции

```
def say_hello():
    print("Привет!")
    print("Приятно познакомиться!")

# Использовать функцию
say_hello()
say_hello()

# Вывод:
# Привет!
# Приятно познакомиться!
```

```
# Привет!  
# Приятно познакомиться!
```

Части функции:

```
def function_name():    # def, имя, (), и :  
    # Код с отступом  
    print("Внутри функции")
```

Функции с параметрами

Параметры позволяют передавать информацию в функцию:

```
def greet(name):  
    print(f"Привет, {name}!")  
  
greet("Ева")          # Выведет: Привет, Ева!  
greet("Робот")        # Выведет: Привет, Робот!
```

Несколько параметров:

```
def run_motor(speed, duration):  
    motor.run(speed)  
    time.sleep(duration)  
    motor.stop()  
  
# Использовать  
run_motor(30, 3)      # Вращать со скоростью 30 об/мин в течение 3 секунд  
run_motor(60, 5)      # Вращать со скоростью 60 об/мин в течение 5 секунд
```

Функции, которые возвращают значения

Функции могут возвращать результат:

```
def add(a, b):  
    result = a + b  
    return result  
  
total = add(5, 3)  
print(total)        # Выведет: 8
```

Примеры с роботом:

```
def pulses_to_degrees(pulses):  
    degrees = pulses / 8.889  
    return degrees  
  
position = motor.get_position()  
degrees = pulses_to_degrees(position)  
print(f"Мотор на {degrees:.1f} градусах")
```

```
def is_motor_moving(motor):
    speed = motor.get_speed()
    if speed != 0:
        return True
    else:
        return False

if is_motor_moving(motor1):
    print("Мотор 1 движется!")
```

Зачем использовать функции?

1. Избегать повторений:

```
# Без функции - много повторений
motor1.start()
motor1.run(30)
time.sleep(2)
motor1.stop()

motor2.start()
motor2.run(30)
time.sleep(2)
motor2.stop()

# С функцией - написать один раз, использовать много раз
def test_motor(motor):
    motor.start()
    motor.run(30)
    time.sleep(2)
    motor.stop()

test_motor(motor1)
test_motor(motor2)
```

2. Делать код читаемым:

```
# Сложно понять
motor.run(30)
time.sleep(2)
motor.run(60)
time.sleep(2)
motor.run(30)
time.sleep(2)
motor.stop()

# Легко понять
def speed_test():
    motor.run(30)      # Начать медленно
    time.sleep(2)
    motor.run(60)      # Ускориться
    time.sleep(2)
    motor.run(30)      # Замедлиться
    time.sleep(2)
```

```
motor.stop()

speed_test()
```

3. Организовывать сложный код:

```
def setup_robot():
    motor1.start()
    motor2.start()
    motor3.start()
    motor4.start()

def drive_forward():
    motor1.run(40)
    motor2.run(40)
    motor3.run(40)
    motor4.run(40)

def stop_robot():
    motor1.stop()
    motor2.stop()
    motor3.stop()
    motor4.stop()

# Главная программа - очень понятно!
setup_robot()
drive_forward()
time.sleep(3)
stop_robot()
```

Импорт - использование инструментов

Import позволяет использовать код, который написал кто-то другой.

Базовый импорт

```
import time

time.sleep(2)    # Использовать функцию sleep из модуля time
```

Импорт из EvaBot

```
from evabot import Motor

motor = Motor(1)
```

Что это значит: - from evabot - Из пакета evabot - import Motor - Получить инструмент Motor

Распространённые импорты для робототехники

```
# Всегда нужны эти
from evabot import Motor
import time

# Создать моторы
motor1 = Motor(1)

# Использовать time.sleep()
time.sleep(3)
```

Импорт нескольких вещей

```
from evabot import Motor, Robot

motor = Motor(1)
robot = Robot()
```

Время - ожидание и задержки

Модуль `time` помогает управлять временем в твоих программах.

`time.sleep()`

Заставить программу подождать:

```
import time

print("Начало")
time.sleep(2)          # Подождать 2 секунды
print("2 секунды спустя")
```

Использование с роботом:

```
motor.run(30)
time.sleep(3)          # Позволить мотору вращаться 3 секунды
motor.stop()
```

Дробные секунды:

```
time.sleep(0.5)        # Подождать полсекунды
time.sleep(0.1)        # Подождать 1/10 секунды
time.sleep(2.5)        # Подождать 2.5 секунды
```

Собираем всё вместе

Вот полная программа для робота, использующая всё, что ты выучил:

```
# Импортировать инструменты
```

```

from evabot import Motor
import time

# Функция для тестирования мотора
def test_motor(motor_id, duration):
    print(f"Тестируем мотор {motor_id}...")

    # Создать и запустить мотор
    motor = Motor(motor_id)
    motor.start()

    # Вращать на разных скоростях
    speeds = [20, 40, 60, 40, 20]

    for speed in speeds:
        print(f" Скорость: {speed} об/мин")
        motor.run(speed)
        time.sleep(duration)

    # Проверить, как далеко проехал
    position = motor.get_position()
    rotations = position / 3200
    print(f" Прошёл {rotations:.2f} оборотов")

    # Остановить мотор
    motor.stop()
    print(f"Тест мотора {motor_id} завершён!\n")

# Главная программа
print("=== Программа тестирования моторов ===\n")

# Протестировать моторы 1 и 2
test_motor(1, 1)
test_motor(2, 1)

print("Все тесты завершены!")

```

Эта программа использует: - □ Комментарии - □ Переменные - □ Функции - □ Циклы for - □ Математику - □ F-строки - □ Импорт - □ Условия if (можно добавить)

Частые ошибки новичков

1. Забыли двоеточие

```

# НЕПРАВИЛЬНО
if speed > 50
    print("Быстро")

# ПРАВИЛЬНО
if speed > 50:
    print("Быстро")

```

Всегда ставь : после: if, elif, else - for, while - def function_name()

2. Использование = вместо ==

```
# НЕПРАВИЛЬНО - это присваивает 50 переменной speed
if speed = 50:
    print("Пятьдесят")
```

```
# ПРАВИЛЬНО - это проверяет, равна ли speed 50
if speed == 50:
    print("Пятьдесят")
```

3. Неправильные отступы

```
# НЕПРАВИЛЬНО
if speed > 50:
print("Быстро")    # Без отступа!
```

```
# ПРАВИЛЬНО
if speed > 50:
    print("Быстро")
```

4. Опечатки в именах переменных

```
# НЕПРАВИЛЬНО
speed = 30
print(Speed)    # Заглавная S - Python думает, что это другая переменная!
```

```
# ПРАВИЛЬНО
speed = 30
print(speed)    # Одинаковое написание
```

5. Забыли скобки

```
# НЕПРАВИЛЬНО
print "Привет"    # Нет ()
```

```
# ПРАВИЛЬНО
print("Привет")
```

6. Деление на ноль

```
# НЕПРАВИЛЬНО - вызовет ошибку!
result = 10 / 0
```

```
# ПРАВИЛЬНО - сначала проверить
if divisor != 0:
    result = 10 / divisor
else:
    print("Нельзя делить на ноль!")
```

Упражнения для практики

Попробуй их, чтобы попрактиковать то, что выучил!

Упражнение 1: Разгон скорости

Напиши программу, которая постепенно увеличивает скорость мотора от 10 до 100 об/мин с шагом 10.

Решение

```
from evabot import Motor
import time

motor = Motor(1)
motor.start()

for speed in range(10, 101, 10):
    print(f"Скорость: {speed} об/мин")
    motor.run(speed)
    time.sleep(1)

motor.stop()
```

Упражнение 2: Проверка мотора

Напиши функцию, которая проверяет, прошёл ли мотор больше 1000 импульсов. Выведи "Прошёл далеко!", если да, "Не далеко", если нет.

Решение

```
def check_movement(motor):
    position = motor.get_position()
    if position > 1000:
        print("Прошёл далеко!")
    else:
        print("Не далеко")

motor.start()
motor.run(30)
time.sleep(2)
check_movement(motor)
motor.stop()
```

Упражнение 3: Обратный отсчёт

Заставь мотор вращаться, считая в обратном порядке от 5 до 1, затем выведи "Поехали!" и останови его.

Решение

```
from evabot import Motor
import time
```

```
motor = Motor(1)
motor.start()
motor.run(30)

for i in range(5, 0, -1):
    print(i)
    time.sleep(1)

print("Поехали!")
motor.stop()
```

Краткая справка

Обязательно запомни

```
# Импорт
from evabot import Motor
import time

# Переменные
speed = 30
name = "Робот"

# Вывод
print("Привет")
print(f"Скорость: {speed}")

# Условие if
if speed > 50:
    print("Быстро")
else:
    print("Медленно")

# Цикл for
for i in range(5):
    print(i)

# Цикл while
count = 0
while count < 5:
    print(count)
    count = count + 1

# Функция
def my_function(parameter):
    print(parameter)
    return result

# Комментарии
# Это комментарий
```

Правила отступов

- Используй 4 пробела (или 1 tab)
- Делай отступ для кода, который находится "внутри" чего-то
- Сохраняй одинаковый отступ для кода на одном уровне
- Не смешивай пробелы и табуляцию!

Распространённые символы

```
=      # Присвоить (speed = 30)
==     # Проверить на равенство (if speed == 30)
!=     # Не равно (if speed != 30)
>      # Больше чем
<      # Меньше чем
>=     # Больше или равно
<=     # Меньше или равно
+      # Сложить
-      # Вычесть
*      # Умножить
/      # Разделить
#      # Комментарий
:      # Начать блок (if/for/def)
()      # Вызов функции или параметры
[]     # Список
```

Следующие шаги

Теперь, когда ты знаешь основы Python, ты готов программировать роботов!

Переходи к: [Глава 1: Начало работы](#)

Ты будешь использовать всё, что выучил здесь: - Переменные для хранения скорости моторов - Условия if для принятия решений - Циклы для повторения движений - Функции для организации кода - Импорт для использования класса Motor

Удачи в программировании твоего робота!